



Parallel Models & Conditions of Parallelism

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year II Semester

ACADEMIC YEAR : 2025 - 2026

Theory of Parallelism & Models

Parallel Computing aims to execute multiple calculations simultaneously. In computer architecture, we analyze parallelism using standardized parallel models to design efficient hardware configurations.

Parallel Computer Models

- **Multiprocessors (Shared Memory):** Multiple CPUs sharing a single physical memory address space (e.g. SMP systems).
- **Multicomputers (Distributed Memory):** Nodes with private memory communicating via message passing (e.g. clusters).
- **SIMD Computers:** Single Instruction, Multiple Data. One controller executes a single command on vectors of data (e.g. GPUs).
- **PRAM Model:** Parallel Random Access Machine. A theoretical model to analyze parallel algorithm complexities.

Program Partitioning & Flow

- **Parallelism Granularity:** Sizes of execution blocks. Ranges from fine-grained (instruction level) to coarse-grained (process level).
- **Static Scheduling:** Compiler assigns instructions to cores at compilation time. Low runtime overhead.
- **Dynamic Scheduling:** Hardware assigns tasks at runtime based on resource availability, handling hazards dynamically.
- **Control vs. Data Flow:** Control-flow: sequential, instruction-pointer driven. Data-flow: instructions execute as soon as their operands are ready.

Bernstein's Conditions of Parallelism

A set of algebraic conditions used to determine if two software statements can execute in parallel:

Mathematical Formulation

Let I1 and I2 be two program statements.

- **Read Set (R_i):** Set of all variables read by statement I_i.
- **Write Set (W_i):** Set of all variables modified by statement I_i.

Statements I1 and I2 can execute in parallel (I1 || I2) if and only if they satisfy three conditions:

1. $R1 \cap W2 = \emptyset$ (No Flow Dependency)
2. $W1 \cap R2 = \emptyset$ (No Anti-Dependency)
3. $W1 \cap W2 = \emptyset$ (No Output Dependency)

Dependency Tracing Example

-- Assume two statements:

I1: A := B + C;

I2: D := A * E;

• Read/Write Sets:

R1 = {B, C} W1 = {A}

R2 = {A, E} W2 = {D}

• Check Bernstein's Rules:

$R1 \cap W2 = \emptyset$ ✓

$W1 \cap R2 = \{A\}$ ✗ (Flow Dependency)

$W1 \cap W2 = \emptyset$ ✓

- **Conclusion:** I1 and I2 CANNOT run in parallel. I2 must wait for I1.

System Interconnect Architectures

Parallel systems use different network topologies to route data between processors and memory:

1. Shared Bus Network

- Description: A single shared communication line connects all processors and memory modules.
- Cost: Very low.
- Bottleneck: Serious bandwidth congestion as node count scales past 8-16 processors.

2. Crossbar Switch

- Description: A matrix of grid switches connecting every processor directly to every memory module.
- Performance: Maximum bandwidth; zero blocking conflicts.
- Cost: $O(N^2)$ switches — very expensive for large systems.

3. Multistage Networks (MINs)

- Description: Uses stages of small 2x2 switches (like Omega or Butterfly networks) to route packets.
- Trade-off: Balanced cost $O(N \log N)$ and good scaling throughput. Standard in supercomputer grids.

Applications of Parallel Models

Parallel architectures and dependency models drive modern high-performance systems:

Optimizing Compilers

Compilers apply Bernstein's conditions to construct dependency graphs of code instructions. Allows compilers to schedule instructions to run in parallel on multicore CPUs safely.

GPU Vector Processing

Modern graphics cards partition image rendering logic into millions of vector threads. Executing them in parallel speeds up real-time 3D games.

Distributed Supercomputing

Supercomputers (like weather models) partition computations across thousands of nodes using MPI libraries to solve problems in days instead of years.

Database Query Partitioning

Cloud databases partition tables and queries. Running lookups in parallel across nodes decreases query latencies under load.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management